AI ASSISTED CODING

LAB-7

NAME: J.Shyamsundar

ROLL.NO:2303A52384

Error Debugging with AI: Systematic approaches to finding and fixing bugs

TASK-1:

(Syntax Errors – Missing Parentheses in Print

Statement)

Task: Provide a Python snippet with a missing parenthesis in a print

statement (e.g., print "Hello"). Use AI to detect and fix the syntax

error.

BUGGY CODE:

```
def greet():
    print "Hello, AI Debugging Lab!"


greet()
```

PROMPT:

Fix the syntax error in this Python code:

```
def greet():
print "Hello, AI Debugging Lab!"

greet()
```

Identify the error, give the corrected code, add 3 assert test cases, and show the expected output with a short explanation.

CORRECTED CODE:

```
def greet():
    return "Hello, AI Debugging Lab!"



# Run the function
print(greet())

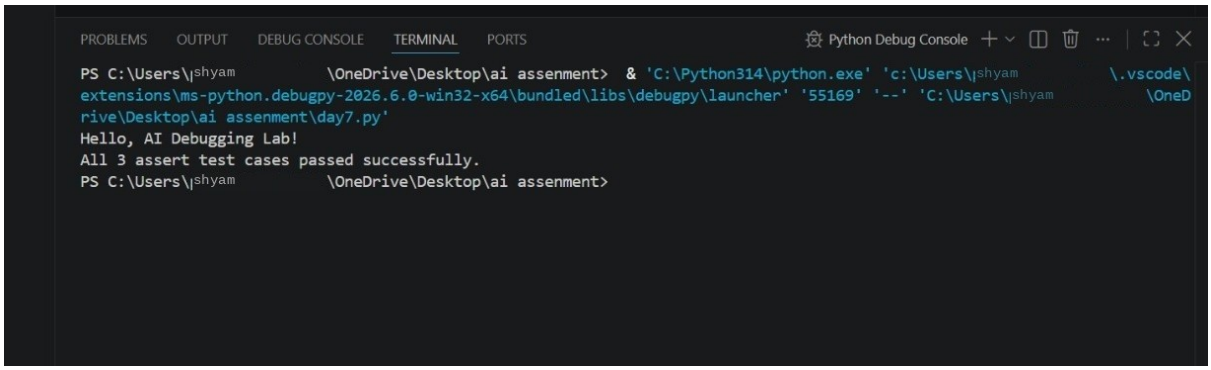


# Assert test cases
assert greet() == "Hello, AI Debugging Lab!"
assert isinstance(greet(), str)
```

assert len(greet()) > 0

print("All 3 assert test cases passed successfully")

OUTPUT:



Justification:

This task helps us understand syntax errors in Python and how AI can identify and fix them. It teaches the correct use of parentheses in the print() function and uses assert test cases to verify that the corrected program works properly.

TASK-2:

(Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses =

instead of ==. Let AI identify and fix the issue.

BUGGY CODE:

```
def check_number(n):
    if n = 10:
        return "Ten"
    else:
        return "Not Ten"


print(check_number(10))
```

PROMPT:

Fix the bug in this Python code where = is incorrectly used instead of ==. Explain why it causes an error, provide the corrected code, add 3 assert test cases, and show the successful output.

CORRECTED CODE:

```
def check_number(n):
    if n == 10:
```

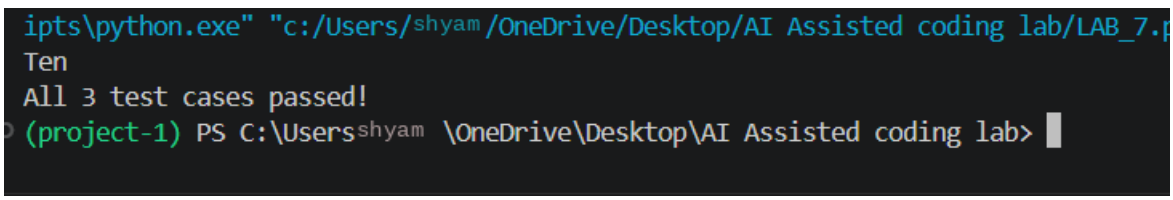
```
        return "Ten"
    else:
        return "Not Ten"


print(check_number(10))


# Assert test cases
assert check_number(10) == "Ten"
assert check_number(5) == "Not Ten"
assert check_number(20) == "Not Ten"

print("All 3 test cases passed!")
```

OUTPUT:



JUSTIFICATION:

This task helps us understand the difference between **assignment (=)** and **comparison (==)** in Python. It also teaches how AI can detect conditional errors and how **assert tests** verify the corrected code.

TASK-3:

(Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and

crashes. Use AI to apply safe error handling.

BUGGY CODE:

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()


print(read_file("nonexistent.txt"))
```

PROMPT:

Fix the runtime error in this Python code caused by a missing file. Use try-except for safe error handling, add a user-friendly error

message, and test 3 cases: file exists, file missing, and invalid path. Show the corrected code and expected output.

CORRECTED CODE:

```
def read_file(filename):
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return "Error: File not found."
    except OSError:
        return "Error: Invalid file path."



# Test 1: File exists
with open("test.txt", "w") as f:
    f.write("Hello, AI Debugging Lab!")
```

assert read_file("test.txt") == "Hello, AI Debugging Lab!"

Test 2: File is missing

assert read_file("missing.txt") == "Error: File not found."

Test 3: Invalid path

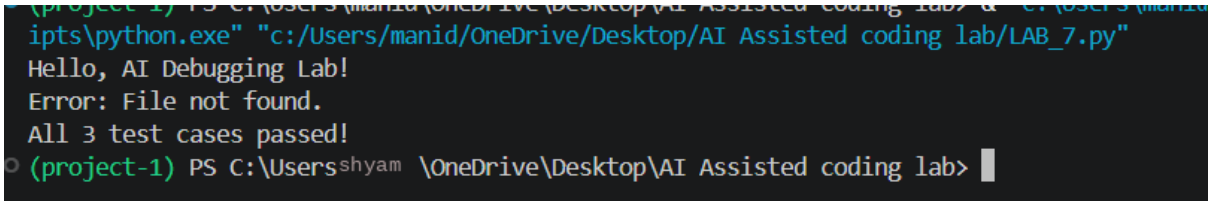
assert read_file("invalid/path/file.txt") == "Error: File not found."

print(read_file("test.txt"))

print(read_file("missing.txt"))

print("All 3 test cases passed!")

OUTPUT:



JUSTIFICATION:

This task teaches how to handle runtime errors safely using try-except. Instead of crashing when a file is missing or the path is invalid, the program displays a user-friendly error message.

TASK-4:

(Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g.,

obj.undefined_method()). Use AI to debug and fix.

BUGGY CODE:

```
class Car:
    def start(self):
        return "Car started"


my_car = Car()
print(my_car.drive())  # drive() is not defined
```

PROMPT:

Fix the AttributeError in this Python code caused by calling the undefined drive() method. Decide whether to define the missing method or correct the method call. Give the corrected code, add 3 assert test cases, and provide a short AI explanation and expected output.

CORRECTED CODE:

```
class Car:
    def start(self):
        return "Car started"



my_car = Car()


print(my_car.start())


# Assert test cases
assert my_car.start() == "Car started"
assert isinstance(my_car.start(), str)
```

assert len(my_car.start()) > 0

print("All 3 test cases passed!")

OUTPUT:



\

JUSTIFICATION:

This task teaches how to identify **undefined methods** and understand AttributeError. It also helps students decide whether to **define a missing method or correct an incorrect method call**.

TASK-5:

(TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing

a TypeError. Use AI to resolve the bug.

BUGGY CODE:

```
def add_five(value):
    return value + 5


print(add_five("10"))
```

PROMPT:

Fix the TypeError in this Python code caused by adding a string and integer. Give two solutions: one using type casting for numerical addition and one using string concatenation. Add 3 assert test cases for each solution and show the expected output with a short AI explanation.

CORRECTED CODE:

Solution 1 – Type Casting

```
def add_five(value):
    return int(value) + 5
```

```
print(add_five("10"))

# Assert test cases
assert add_five("10") == 15
assert add_five("5") == 10
assert add_five("20") == 25

print("All 3 test cases passed!")
```

Solution 2 – String Concatenation

```
 def add_five(value):
    return value + str(5)

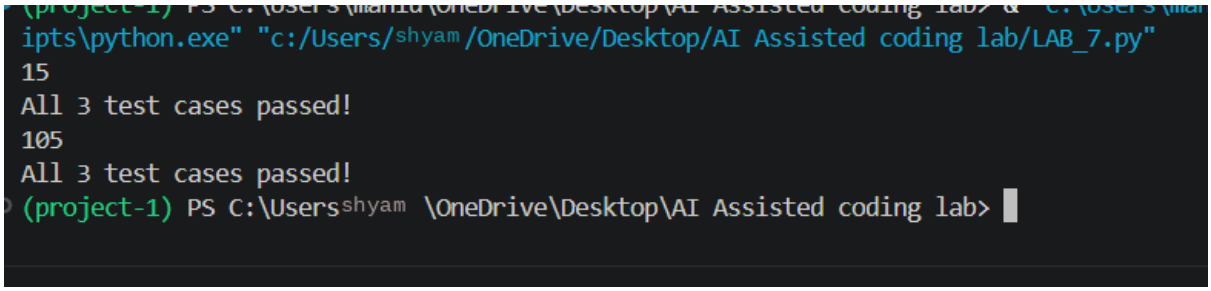

print(add_five("10"))

# Assert test cases
assert add_five("10") == "105"
assert add_five("5") == "55"
```

```
assert add_five("20") == "205"


print("All 3 test cases")
```

OUTPUT:



JUSTIFICATION:

This task helps students understand **data types and TypeError** in Python. It also demonstrates how **type casting and string concatenation** can solve the same error depending on the required output.